6.5 Share controller setup through inheritance

DefaultController and ProductController now repeat the same Twig setup. Put that shared initialization in an abstract Controller superclass. Application, Product, the entry script, templates, and stylesheet remain unchanged.

Create the shared superclass

Create `src/Tutorials/Class6/Controller.php`:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class6;

use Twig\Environment;
use Twig\Loader\FilesystemLoader;

abstract class Controller
{
    private const PATH_TO_TEMPLATES = __DIR__ .
'/../../../templates/tutorials/class6';

    protected Environment $twig;

    public function __construct()
    {
        $loader = new FilesystemLoader(self::PATH_TO_TEMPLATES);
        $this->twig = new Environment($loader, [
            'autoescape' => 'html',
            'strict_variables' => true,
        ]);
    }
}
```

abstract prevents `new Controller()`. It communicates that this class supplies common behavior for concrete page controllers, rather than representing a page on its own. An abstract class can contain implemented methods and does not need an abstract method.

The protected twig property is accessible inside Controller and its subclasses, but not directly from Application. A private parent property would not be accessible to the child page methods. The template-path constant stays private because only the constructor defined here needs it.

Replace both page controllers

Replace `src/Tutorials/Class6/DefaultController.php` with:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class6;

class DefaultController extends Controller
{
    public function home(): void
    {
        $template = 'home.html.twig';
        $args = [];
        echo $this->twig->render($template, $args);
    }

    public function contact(): void
    {
        $template = 'contact.html.twig';
        $args = [];
        echo $this->twig->render($template, $args);
    }
}
```

Replace `src/Tutorials/Class6/ProductController.php` with:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class6;

class ProductController extends Controller
{
    public function productList(): void
    {
        $products = [
            new Product('Study notebook', 4.50),
            new Product('Desk organizer', 12.00),
            new Product('Pencil set', 3.25),
        ];

        $template = 'products.html.twig';
        $args = ['products' => $products];
        echo $this->twig->render($template, $args);
    }
}
```

Neither subclass defines a constructor, so each inherits Controller's constructor. Constructing a DefaultController or ProductController still initializes that object's Twig environment. Inheritance shares the setup code; it does not turn the two controllers' twig properties into one shared static object.

Keep the action methods public: Application still calls them from outside the controller hierarchy. Remove duplicated Twig imports, constants, properties, and constructors from the children using the full replacements above. If a child defines its own constructor, it must call `parent::__construct()` to retain this setup. We will do exactly that in 6.6 to initialize a collection shared by two product actions.

Verify behavior and explain the two kinds of inheritance

Repeat all route and empty-list checks from 6.4. The HTML should be unchanged. Inspect the PHP files: only Controller now contains the Twig loader and environment setup.

PHP's `extends Controller` shares PHP properties and methods. Twig's `extends 'base.html.twig'` shares the document layout and blocks. They solve similar duplication problems in different layers; neither declaration substitutes for the other.